WiDS 2025: Image Deblurring using Deep Learning

Final Project Report

Hari Sheth
Mentor: Sam Navdeep
Department: CSE
IIT Bombay
https://github.com/Hari-Sheth/WiDS-2025

February 1, 2026

Abstract

The project was on Image deblurring and it started from the basics of Neural Networks(classifying handwritten digits using MNIST dataset), proceeding towards Convolutional Neural Networks for image classification, and then towards SRCNN and Generative Adversarial Networks for the final submission.

Contents

Week 1: Basics of Neural networks **2**
- Task . . . 2
- Model Design and Architecture . . . 3
  - Model Architecture . . . 3
  - Training Procedure . . . 3
  - Evaluation on Test Data . . . 3
- Classifying Hand-written digits using a CNN . . . 4
  - Architecture . . . 4

Week 2: Image Classification **5**
- Task . . . 5
- Model Design and Architecture . . . 5

Week 3: The beginning of Image deblurring **6**
- Why Images get Blurred? . . . 6
- Common Image Interpolation Techniques . . . 7
- SRCNNs for Image Deblurring . . . 7
- Model Design . . . 8
  - Architecture . . . 8
  - MSE Loss . . . 8
  - PSNR Value . . . 8

Week 4: Using GANs for Image Deblurring **9**
- What are GANs? . . . 9
- Generator design . . . 10
- Discriminator design . . . 10
- Results & Conclusion . . . 10

Week 1: Basics of Neural networks

Task

Using a fully connected Neural network, identify handwritten digits using the MNIST dataset to train on.

Learning

The week consisted of many YouTube lectures and online material provided by the mentor to find out how neural networks work.
I got an understanding of what neural networks are and gradient descent etc. in neural networks. However, the model did not give the expected results. For example, on tuning the hyperparameters, the model would only predict 5 or 3 for all the self drawn digits. Then I realized that MNIST images have a black background and white digits, and some transformations must also be done to the image to improve prediction. In doing this, I got a little better accuracy for my own written digits. One problem I was not able to figure out was that the model showed a really great accuracy on the test dataset, but not on my self written digits(even after the image editing).
Meanwhile, as self-learning I also wrote a code that outputs slope and intercept for a given set of x and y coordinates of the best-fit line.
At the end of week 1 my mentor enlightened me what are CNNs and how would they work better(led me to my first side-quest)

Model Design and Architecture

Model Architecture

The fully connected neural network had three hidden layers and it was implemented using PyTorch. The network architecture consists of:

An input layer of size 784 (flattened image)

A first hidden layer with 256 neurons

A second hidden layer with 128 neurons

An output layer with 10 neurons corresponding to the digit classes (0–9)

Batch normalization[1] was applied to stabilize training and dropout[2] of 0.2. The final layer outputs raw tensor, which was used for classification.

Training Procedure

The loss function was Cross-Entropy loss function, suitable for classification tasks. Optimization was performed using the Adam optimizer. number of epochs was set to 40, and the loss was monitored across epochs to ensure convergence. As the part of assignment, I had to find out the effect of different hyperparameters like learning rate and batch size.

The performance of the model showed that larger batch sizes usually resulted in lower training loss, while accuracy remained relatively stable across configurations. Also, an increased batch size resulted in faster training.

Evaluation on Test Data

As the final part of satisfaction digits were manually written on paper and resized, inverted, and transformed to MNIST like images, however, the performance here was not up to the mark.
Hence that started my second side quest of using a CNN.

[1]Batch normalization is a technique that normalizes layer inputs during training to stabilize learning, accelerate convergence, and improve overall model performance.

[2]Dropout is a regularization technique in neural networks that randomly deactivates a subset of neurons(perceptrons) during training in order to reduce overfitting and prevent the model from becoming overly dependent on specific neurons. By randomly disabling different neurons in each training iteration, dropout encourages the network to learn more generalized feature representations.

Classifying Hand-written digits using a CNN[3]

The only changes to the model architecture were made while keeping the loss function and optimizer the same, resulting in much better performance.

Architecture

The model consisted of two Convolutional layers using ReLU for non-linearization and two pooling layers[4] to reduce training time. The model has been shown below.

```
import torch
import torch.nn as nn
class CNN(nn.Module):
    def __init__(self, in_channels = 1, num_classes = 10):
        super(CNN, self).__init__()
        self.conv1 = nn.Conv2d(in_channels = 1, out_channels = 8,
           kernel_size = (3,3), stride = (1,1), padding = (1,1))
        self.pool = nn.MaxPool2d(kernel_size = (2,2), stride = (2,2))
        self.conv2 = nn.Conv2d(in_channels = 8, out_channels = 16,
           kernel_size = (3,3), stride = (1,1), padding = (1,1))
        self.fc1 = nn.Linear(16*7*7, num_classes)
    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = self.pool(x)
        x = F.relu(self.conv2(x))
        x = self.pool(x)
        x = x.reshape(x.shape[0], -1)
        x = self.fc1(x)

        return x
```

[3]CNN stands for Convolutional Neural Networks. It is useful in image classification as it looks at the image patch by patch instead of converting it to a one-dimensional tensor and looking at the raw numbers. This makes the model much more efficient as the image data is also patch dependent, and would not show up its beauty when the 2D image is converted into a straight line

[4]Pooling layers are used in Convolutional Neural Networks to reduce the spatial dimensions of feature maps, which helps decrease computation, control overfitting, and make the network more translation-invariant. Pooling layers, such as max pooling and average pooling, summarize regions of the input feature map by taking the maximum or average value, respectively, reducing width and height while retaining important features.

Week 2: Image Classification

Task

The task was to classify an image classification dataset containing at least five classes and RGB images using a Convolutional Neural Network (CNN). For this purpose, the Intel Image Classification Dataset[5] was selected, with class labels belonging to the set {*buildings, forest, glacier, mountain, sea, street*}.

Model Design and Architecture

Model consisted of 3 Convolutional layers each followed by a Max-Pooling layer, and finally a Linear layer and dropout of 0.2. The loss function was again Cross-Entropy loss and optimizer was Adam The interesting point here is the images were of a tensor of size $[3, 64, 64]$ because the images were color images. The final accuracy was not satisfactory, but the possible reason could be that the classes are not very specific, an image could be of an *mountain* and a *glacier* at the same time, or it might contain a *building* by the *street*, such images cannot be classified into a single class. The model could have been improved by predicting multiple classes for a better accuracy, however the dataset did not provide the provision for multiple indexes.

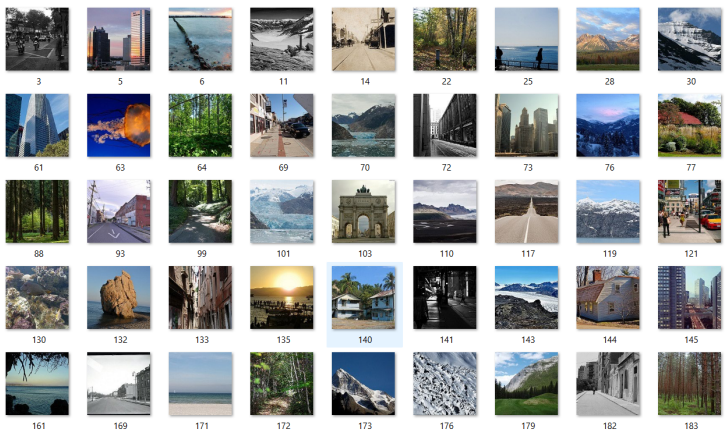


Figure 1: Sample images from the dataset.

[5]Link to the dataset: https://www.kaggle.com/datasets/puneet6060/intel-image-classification

Week 3: The beginning of Image deblurring

Why Images get Blurred?

No scene cannot be perfectly captured by a camera. A number of practical and physical limitations, such as camera shake from hand movements, defocus blur from imperfect focus or the need to image objects at different distances within the same field of view, the limited number of pixels in the image sensor, and random noise introduced during the imaging process, inherently limit the resolution of any image. The amount of detail that can be accurately captured in a photograph is limited by these factors taken together.

Two common image restoration problems are:

1. **Image Deblurring**
   Image deblurring is the process of removing blur from an image caused by camera shake, motion, or defocus. The goal is to restore sharpness and recover lost details, making the image clearer and more visually accurate. Techniques can include inverse filtering, Wiener filtering, and modern deep learning-based methods that estimate the blur and reverse its effect.

2. **Image Super-Resolution**
   Image super-resolution aims to increase the resolution of an image, generating a higher-resolution version from a low-resolution input. This involves predicting or reconstructing missing high-frequency details to create a sharper, more detailed image. Methods range from interpolation-based approaches (like bilinear or bicubic) to advanced deep learning models that learn patterns from large image datasets.

Common Image Interpolation Techniques

Image interpolation is the process of increasing the number of pixels of an image. The simplest techniques for this process are:

1. **Nearest Neighbor (Easy Duplication)**
   Assigning the value of the old pixel to the newly created pixels in its place is the simplest way to enlarge an image. This technique frequently results in a pixelated, blocky image and does not require interpolation.

2. **Linear Interpolation (2D Bilinear)**
   Linear interpolation, which computes the new pixel values as a weighted average of the nearby pixels, is a superior method. Compared to simple duplication, this produces smoother images with more gradual transitions.

3. **Cubic Interpolation (2D Bicubic)**
   By utilizing a 4×4 grid of nearby pixels and fitting a cubic curve to estimate new pixel values, cubic interpolation outperforms linear interpolation. Compared to linear interpolation, this technique yields even smoother images and more accurate edge preservation.

However, all these techniques have one problem, they don't really add new information, which was lost due to the blur.
To overcome these limitations, researchers started using machine learning, and now instead of fixed patterns, models learn by observing data, which include low quality images and corresponding high quality image.

SRCNNs[6] for Image Deblurring

SRCNNs were first technique to directly apply a Convolutional Neural Network to learn an end-to-end mapping between a low-resolution image and its high-resolution counterpart was SRCNN, which was first introduced in 2014.
SRCNN reconstructs fine details and improves image quality by passing an upscaled low-resolution image through a short sequence of convolutional layers rather than using conventional interpolation or patch-based techniques. Patch extraction and representation, nonlinear mapping, and reconstruction are the three primary stages of a network. SRCNN showed that deep learning could greatly outperform conventional super-resolution techniques despite its straightforward architecture, opening the door for more sophisticated models like VDSR, EDSR, and GAN-based techniques.

[6]Super-Resolutional Convolutional Neural Networks

Model Design

Architecture

The convolutional layers have the following parameters:

1. **Convolution Layer 1:** Kernel size: $3 \times 9 \times 9$, Number of filters: 64
2. **Convolution Layer 2:** Kernel size: $64 \times 1 \times 1$, Number of filters: 32
3. **Convolution Layer 3:** Kernel size: $32 \times 5 \times 5$, Number of filters: 3

The loss function is MSE loss and accuracy is found by the PSNR value.

MSE Loss

Mean Squared Error (MSE) Loss

Mean Squared Error (MSE) is a commonly used loss function in regression and image reconstruction tasks. It measures the average squared difference between predicted values and ground truth targets. The squaring operation penalizes larger errors more strongly.

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

where N is the number of samples, y_i is the true value, and $\hat{y}_i$ is the predicted value. MSE is differentiable and well-suited for gradient-based optimization methods.

PSNR Value

Peak Signal-to-Noise Ratio (PSNR)

Peak Signal-to-Noise Ratio (PSNR) is a widely used evaluation metric for image reconstruction and super-resolution tasks. It measures the quality of a reconstructed image compared to its ground truth counterpart, based on the Mean Squared Error (MSE). Higher PSNR values indicate better reconstruction quality.

$$\text{PSNR} = 10 \log_{10} \left(\frac{MAX_I^2}{\text{MSE}} \right)$$

where MAX_I is the maximum possible pixel value of the image (e.g., 255 for 8-bit images), and MSE is the mean squared error between the reconstructed and reference images. PSNR is expressed in decibels (dB). ‘

Week 4: Using GANs for Image Deblurring

What are GANs?

GANs stand for Generative Adversarial Networks, a type of Network where there are two smaller netwoks fighting against each other, one of them is a Generator, and the other is a Discriminator.

1. **Generator:** The job of the Generator is to create sharp images in this case, when it is given a blurred image as input.

2. **Discriminator:** The Discriminator is tasked with identifying the original sharp images, from those created by the Generator.

With each training epoch, both the Generator and the Discriminator get better at what they are doing, and in the ideal end, the Generator would create such perfect fake reconstructions that the Discriminator would not be able to distinguish it from the original image. The dataset used was the GoPro dataset and the loss function and accuracy measure are the same as before.

Making the model work required a lot of debugging, because the model was giving many tensor dimension related errors at first, and after that it was undergoing extremely slow training. After that cleared out, while training, the model frequently stagnated quickly on some loss, so I had to modify the training loop to train the Generator twice for one training of the Discriminator and add a learning rate decay to prevent stagnation.

Generator design

The generator is a skip-connected U-Net encoder–decoder architecture[7] . While the decoder employs transposed convolutions with ReLU, the encoder employs strided 4×4 convolutions with Instance Normalisation and LeakyReLU. To maintain spatial details, feature maps from matching encoder layers are concatenated. A three-channel output with the same resolution as the input is obtained by a final transposed convolution with tanh activation.

Discriminator design

A multi-scale PatchGAN architecture serves as the discriminator. Successive 4×4 convolutional layers with stride 2, Instance Normalization, and LeakyReLU[8] activations make up each PatchDiscriminator[9], generating a patch-wise realism map for a single channel.

Before being sent to the network, the input and target images are concatenated channel-wise. In order to improve both local detail and global consistency assessment, two discriminators operate at different scales: one on the original resolution and one on a downsampled version.

Results & Conclusion

The model improved upon the blur-sharp PSNR value by $4dB$, which is a really big thing on the logarithmic scale. The effects on custom images are not obvious, but can be noticed with some careful observations. Using a better architecture and more tuning could have led to better results, upto $8dB$ of improvment on the blur-sharp PSNR.

— End —

[7] U-Net Generator is an encoder–decoder convolutional architecture with skip connections between corresponding layers that combines low-level details with high-level semantic features to enable precise spatial reconstruction.

[8] LeakyReLU is an activation function defined as $f(x) = \max(\alpha x, x)$, where α is a small constant (e.g., 0.2), allowing a small gradient when $x < 0$ to prevent dead neurons.

[9] PatchDiscriminator (PatchGAN) encourages high-frequency detail and texture realism by classifying overlapping local image patches rather than the entire image.